

Lab 2
Adding a New System Call

TA: Ruei-Lin Gao

Prof.: Hsung-Pin Chang

NCHU System & Network Lab

$ 貼心提醒

- 本次 lab 可能會讓同學身心俱疲 (´;ω;`)
- 實作內容些許複雜，請各位同學耐心跟著步驟
- 過程中如果累了建議去外面呼吸新鮮空氣再回來…



$ Outline

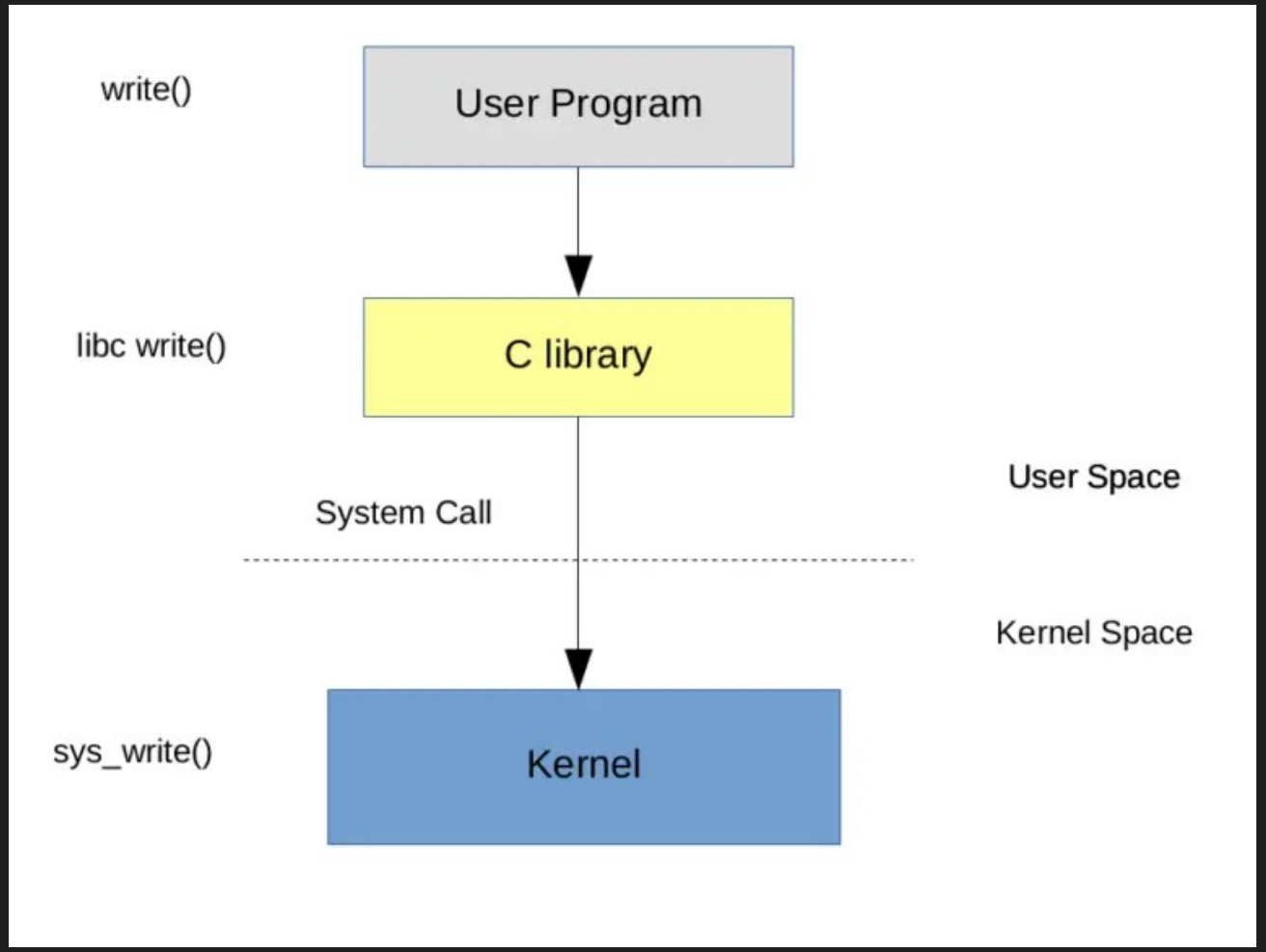
- Introduction
- Get Linux Kernel
- Create a New Syscall
- Compiling and installing
- Testing the new syscall
- Exercise
- Reference

$ Introduction (1/2)

- **What is System call?**
  - A **system call (syscall)** is the mechanism by which a **user-space program requests a service from the operating system's kernel**
- **ex: Process management**: fork(), exec(), exit(), wait()
  - **File operations**: open(), read(), write(), close()



$ Introduction (2/2)

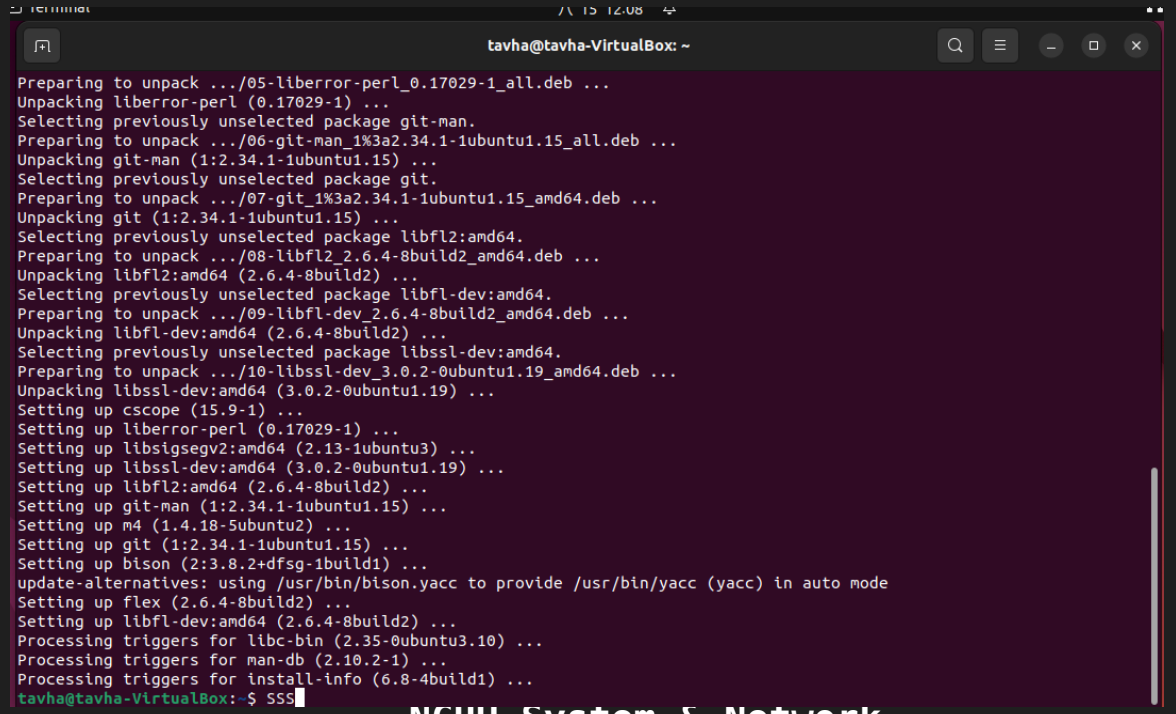


$ Outline

- Introduction
- Get Linux Kernel
- Create a New Syscall
- Compiling and installing
- Testing the new syscall
- Exercise
- Reference

$ Get Linux Kernel(1/3)

- 在 Ubuntu/Debian 系統上安裝編譯 Linux kernel & 開發程式常用的工具 首先在 terminal 輸入” sudo apt update”
  - 接著輸入“ sudo apt-get install build-essential vim git cscope libncurses-dev libssl-dev bison flex”



$ Get Linux Kernel(2/3)

- 下載 Linux kernel(v7.1.3) 原始碼

  輸入” git clone --depth=1 -b v7.1.3 https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git”

```
link@link-VMware-Virtual-Platform:~$ git clone --depth=1 -b v7.1.3 https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git
正拓製到 'linux'...
remote: Enumerating objects: 99207, done.
remote: Counting objects: 100% (99207/99207), done.
remote: Compressing objects: 100% (96454/96454), done.
remote: Total 99207 (delta 7829), reused 21819 (delta 1696), pack-reused 0 (from 0)
接收物件中：100% (99207/99207), 274.35 MiB | 2.21 MiB/s, 完成.
處理 delta 中：100% (7829/7829), 完成.
warning: refs/tags/v7.1.3 113be0295f4f632d61f4e9c93322f70e2365f54d 不是一個提交！
註：切換至「199c9959d3a9b53f346c221757fc7ac507fbac50」。

您正處於「分離 HEAD」狀態。您可以檢視、進行實驗性修改並提交，
而且您可以在切回分支時，捨棄在此狀態下所做的提交
而不對分支造成影響。

如果您想要透過建立分支來保留在此狀態下所做的提交，
您可以現在或稍後在 switch 指令使用 -c 選項。例如：

  git switch -c <新分支名稱>

或者是使用下述命令復原此動作：

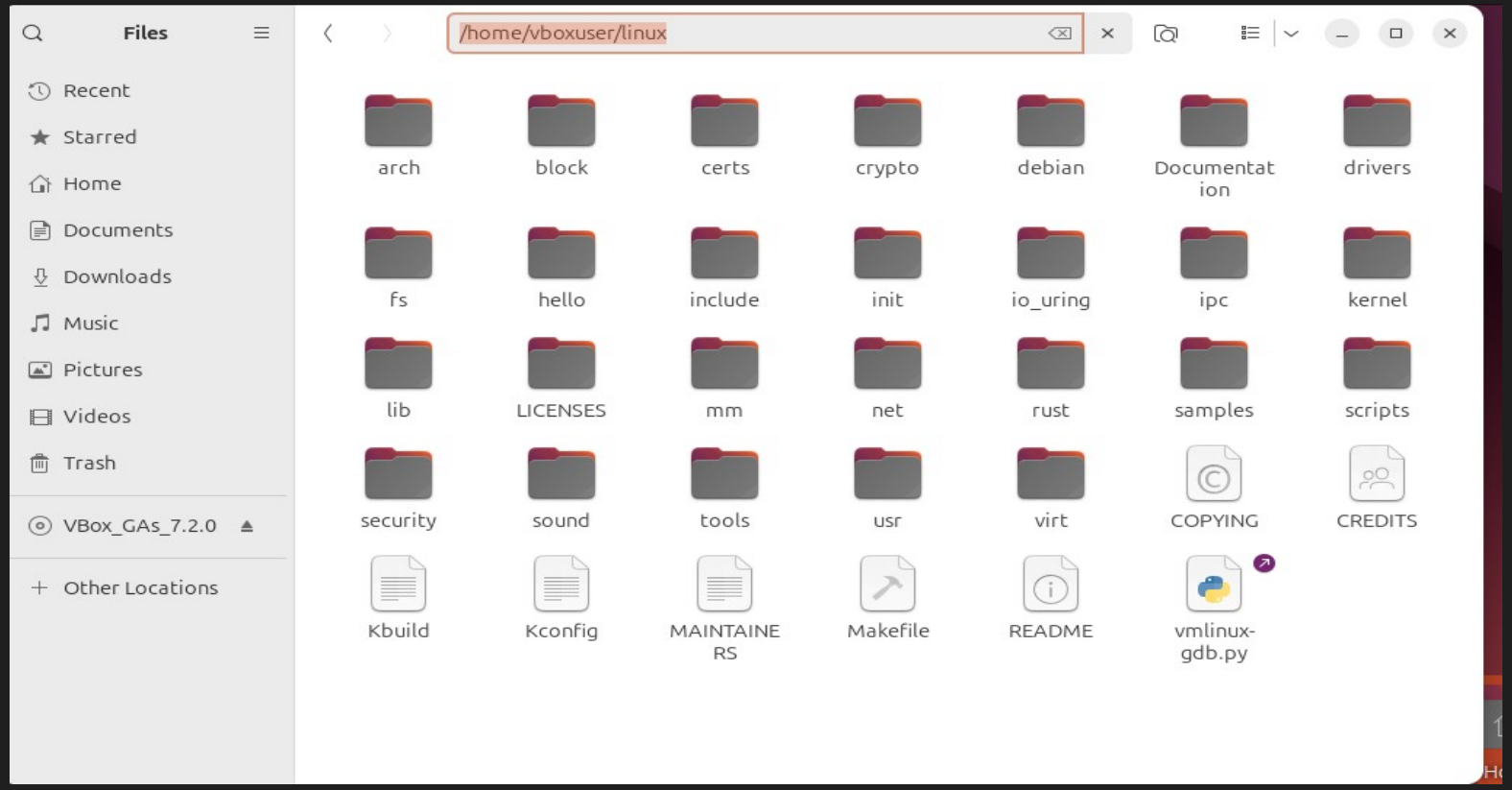
  git switch -

將組態變數 advice.detachedHead 設定為 false，即可關閉本建議
```

$ Get Linux Kernel(3/3)

- 請記住自己下載後 linux 資料夾路徑

  後續動作都要在 linux 資料夾下執行



$ Outline

- Introduction
- Get Linux Kernel
- Create a New Syscall
- Compiling and installing
- Testing the new syscall
- Exercise
- Reference

$ Create a New Syscall (1/8)

- **make ： 自動化編譯工具**
- **Makefile ：** 編譯流程的腳本 ， **make** 根據 **Makefile** 裡寫的規則，決定要執行哪些指令（通常是編譯指令）
- **Makefile** 格式 ：

```
target: dependencies
<TAB> command
```

一定要是 TAB

```
test: test.o
    gcc test.o -o test

test.o: test.c
    gcc -c test.c -o test.o

clean:
    rm -f test test.o
```

$ Create a New Syscall (2/8)

- 輸入以下指令：
  - ✔️ `cd linux`
  - ✔️ `make mrproper`
  - ✔️ `mkdir hello`
  - ✔️ `cd hello`
  - ✔️ `touch hello.c`
  - ✔️ `touch Makefile`

`cd`：切換目錄
`make mrproper`：用於清理，刪掉所有編譯出來的檔案
`mkdir`：創建目錄
`touch`：創建檔案

$ Create a New Syscall (3/8)

- 輸入 open hello.c
- hello.c

```c
#include <linux/kernel.h>
#include <linux/syscalls.h>

SYSCALL_DEFINE0(hello){
  pr_info("Hello\n");
  return 0;
}
```

$ Create a New Syscall (4/8)

- 打開 Makefile 輸入 obj-y += hello.o
- 在 Terminal 輸入

  ✔️ cd ..

  ✔️ cd include/linux/

$ Create a New Syscall (5/8)

- 開啟 syscalls.h
- 把 asmlinkage long sys_hello(void); 加到 syscalls.h 中

```
290 /*
292  * include/uapi/asm-generic/unistd.h. Architecture specific entries go below,
293  * followed by deprecated or obsolete system calls.
294  *
295  * Please note that these prototypes here are only provided for information
296  * purposes, for static analysis, and for linking from the syscall table.
297  * These functions should not be called elsewhere from kernel code.
298  *
299  * As the syscall calling convention may be different from the default
300  * for architectures overriding the syscall calling convention, do not
301  * include the prototypes if CONFIG_ARCH_HAS_SYSCALL_WRAPPER is enabled.
302  */
303 #ifndef CONFIG_ARCH_HAS_SYSCALL_WRAPPER
304+asmlinkage long sys_hello(void)
305 asmlinkage long sys_io_setup(unsigned nr_reqs, aio_context_t __user *ctx);
306 asmlinkage long sys_io_destroy(aio_context_t ctx);
307 asmlinkage long sys_io_submit(aio_context_t, long,
308             struct iocb __user * __user *);
309 asmlinkage long sys_io_cancel(aio_context_t ctx_id, struct iocb __user *iocb,
310                 struct io_event __user *result);
311 asmlinkage long sys_io_getevents(aio_context_t ctx_id,
312                 long min_nr,
313                 long nr,
314                 struct io_event __user *events,
315                 struct __kernel_timespec __user *timeout);
316 asmlinkage long sys_io_getevents_time32(__u32 ctx_id,
317                 __s32 min_nr,
```

$ Create a New Syscall (6/8)

- cd ~/linux
- 開啟 Kbuild 加入 obj-y += hello/

```
93  obj-y              += $(ARCH_LIB)
94  obj-y              += drivers/
95  obj-y              += sound/
96  obj-$(CONFIG_SAMPLES)   += samples/
97  obj-$(CONFIG_NET)    += net/
98  obj-y              += virt/
99  obj-y              += $(ARCH_DRIVERS)
100+ obj-y             += hello/
```

This tells the kernel build system to also compile our newly included folder.

$ Create a New Syscall (7/8)

- Syscall entry
  - Add syscall entry to the architecture-specific table.

    Each CPU architecture could have specific syscalls and

    we need to let them know for which architecture ours is

    made.

    使 user space call syscall 時, 可以被 kernel 接收到

$ Create a New Syscall (8/8)

- Syscall entry (Cont.)
  - cd ~/linux/arch/x86/entry/syscalls/
  - open syscall_64.tbl
  - 加入 462 common hello sys_hello

```
382  458 common  listmount        sys_listmount
383  459 common  lsm_get_self_attr    sys_lsm_get_self_attr
384  460 common  lsm_set_self_attr    sys_lsm_set_self_attr
385  461 common  lsm_list_modules     sys_lsm_list_modules
386+ 462 common  hello               sys_hello
387  #
388  # Due to a historical design error, certain syscalls are numbered
389  # in x32 as compared to native x86_64.  These syscalls have numbe
390  # Do not add new syscalls to this range.  Numbers 548 and above a
391  # for non-x32 use.
```

$ Outline

- Introduction
- Get Linux Kernel
- Create a New Syscall
- Compiling and installing
- Testing the new syscall
- Exercise
- Reference

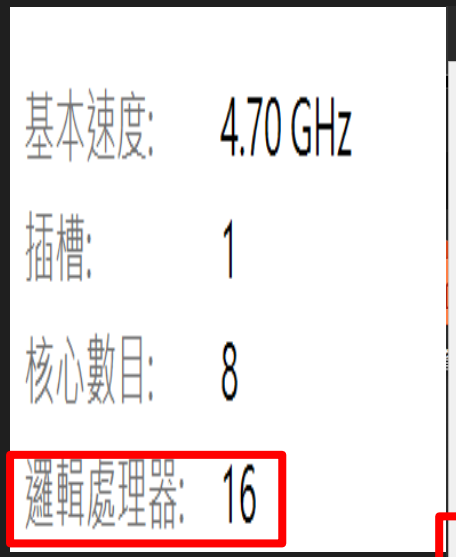
$ Compiling and installing (1/13)

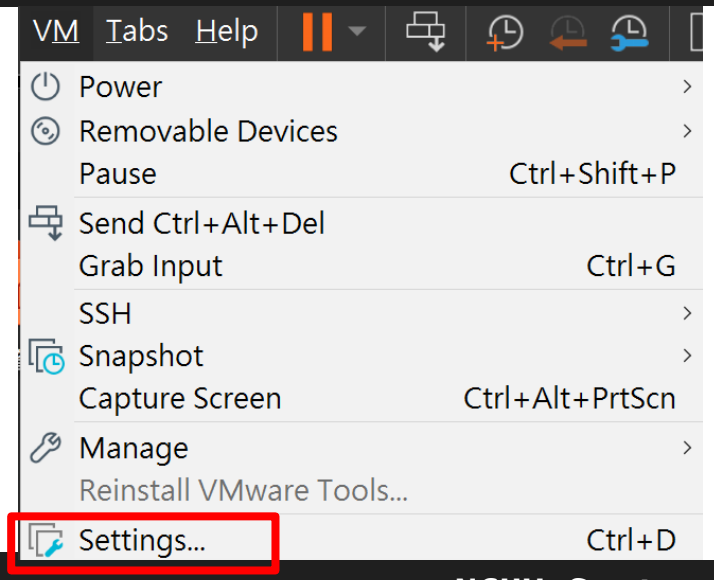
- 設定虛擬機核心數量

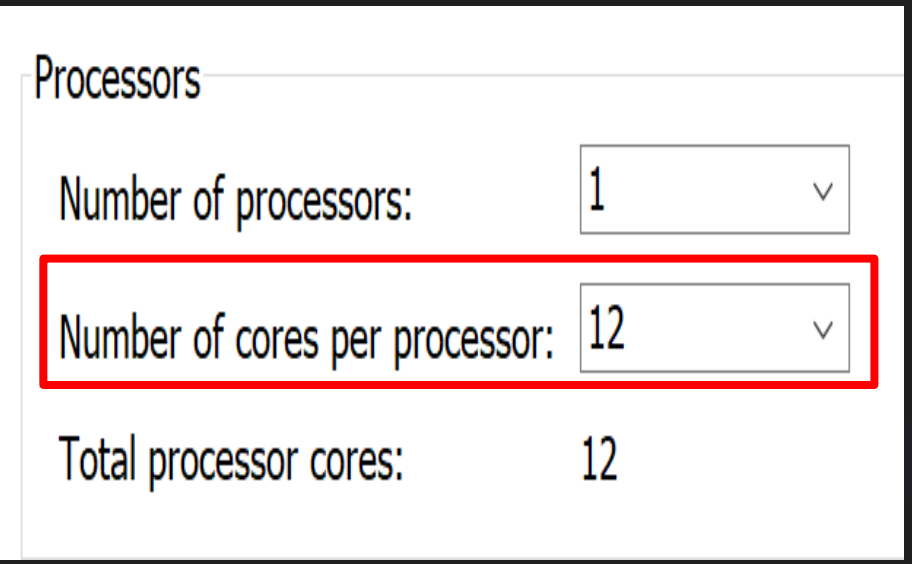
  在 **Windows** 系統中開啟工作管理員（按 **ctrl+alt+del**）查看邏輯處理數量

  在虛擬機管理頁面選擇設定（以 **Vmware** 為例）

  將虛擬機的核心數設置大一點







$ Compiling and installing (2/13)

- Compiling the new kernel

  cd ~/linux 並輸入

  ✔️ cp /boot/config-$(uname -r) .config

  ✔️ make olddefconfig

  ✔️ make localmodconfig

  ✔️ 按 Enter 填上系統預設值即可

```
link@link-VMware-Virtual-Platform:~/linux$ make localmodconfig
using config: '.config'
ghash_clmulni_intel config not found!
System keyring enabled but keys "debian/canonical-certs.pem" not found. Resettin
g keys to default value.
*
* Restart config...
*
*
* PCI GPIO expanders
*
AMD 8111 GPIO driver (GPIO_AMD8111) [N/m/y/?] n
BT8XX GPIO abuser (GPIO_BT8XX) [N/m/y/?] (NEW)
```

$ Compiling and installing (3/13)

- Compiling the new kernel (Cont.)

  ✔️ make -j $(nproc)　　（通常要跑 10+ 分鐘，請耐心等待…

  if 跑完後出現有關 gelf.h 或 dwarf.h 的報錯，執行

  - sudo apt update
  - sudo apt install libelf-dev libdw-dev
  - 再重新執行 make -j $(nproc)

    如有其他報錯可利用 Chatgpt 、 Gemini 查詢解決方法

    亦可找助教討論

$ Compiling and installing (4/13)

- Compiling the new kernel (Cont.)

  ✔️ make -j $(nproc)　　（通常要跑 10+ 分鐘，請耐心等待…

  if 跑完後出現有關 gawk not found 的報錯，執行

  - sudo apt install gawk
  - 再重新執行 make -j $(nproc)

  如有其他報錯可找助教討論

```
link@link-VMware-Virtual-Platform:~/linux$ make -j $(nproc)
  DESCEND objtool
  INSTALL libsubcmd_headers
  GEN     modules.builtin.ranges
/bin/sh: 1: gawk: not found
make[2]: *** [scripts/Makefile.vmlinux:153: modules.builtin.ranges] 錯誤 127
make[2]: *** 正在刪除檔案「modules.builtin.ranges」
make[1]: *** [/home/link/linux/Makefile:1353: vmlinux] 錯誤 2
make[1]: *** 正在等待未完成的作業....
make: *** [Makefile:248: __sub-make] 錯誤 2
```

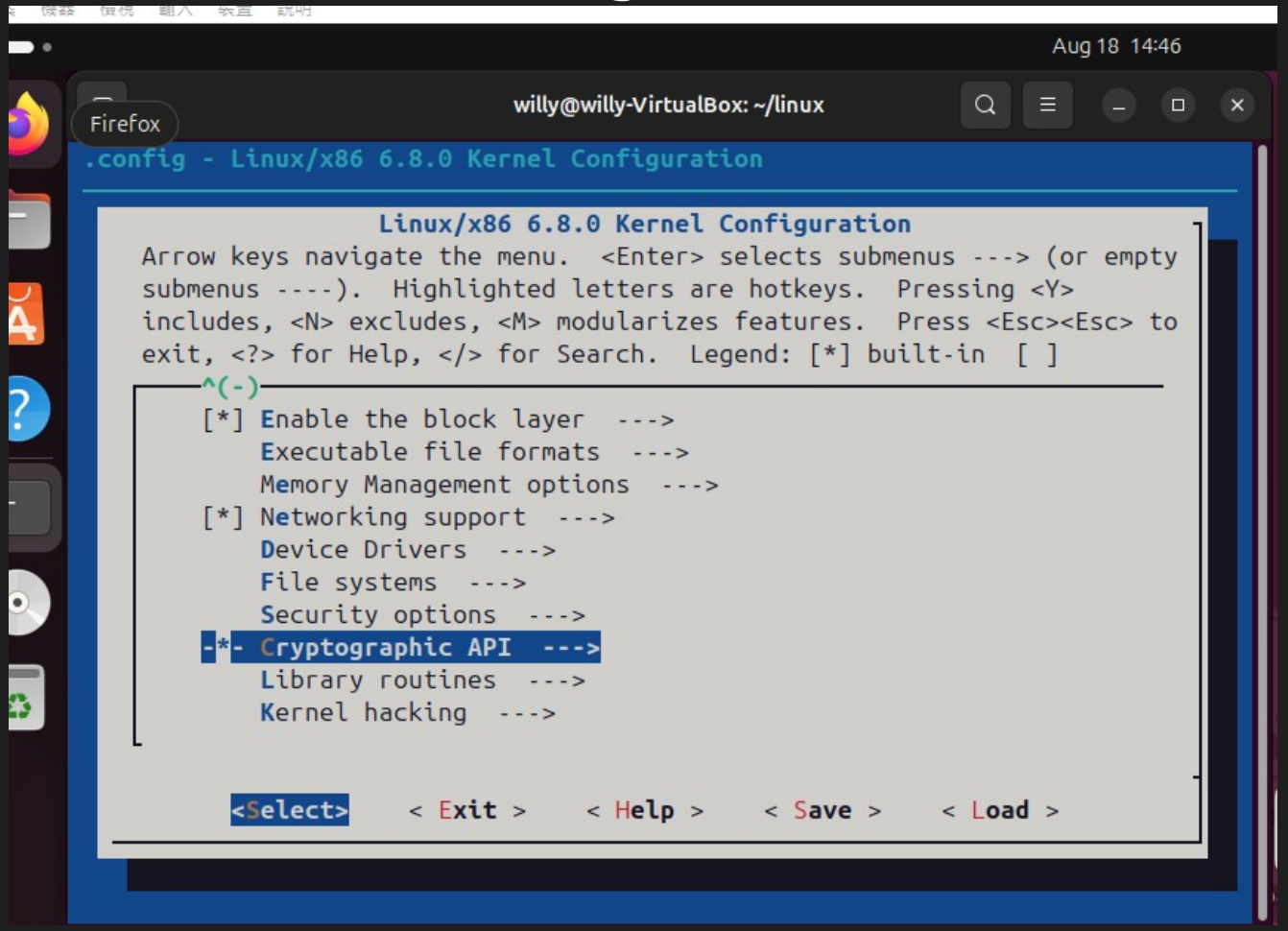
$ Compiling and installing (5/13)

- 若有出現以下 certifacate 相關問題（沒有可跳過到第 12頁）
- 當出現錯誤但訊息太多時可用 make –j1 查看錯誤訊息

```
make[3]: *** No rule to make target 'debian/canonical-certs.pem', needed by 'certs/x509_certificate_list'.  Stop.
make[3]: *** Waiting for unfinished jobs....
  CC      certs/system_keyring.o
make[2]: *** [scripts/Makefile.build:481: certs] Error 2
make[2]: *** Waiting for unfinished jobs....
  AR      kernel/built-in.a
  CC [M]  kernel/kheaders.o
make[1]: *** [/home/willy/linux/Makefile:1921: .] Error 2
make: *** [Makefile:240: __sub-make] Error 2
willy@willy-VirtualBox:~/linux$
```

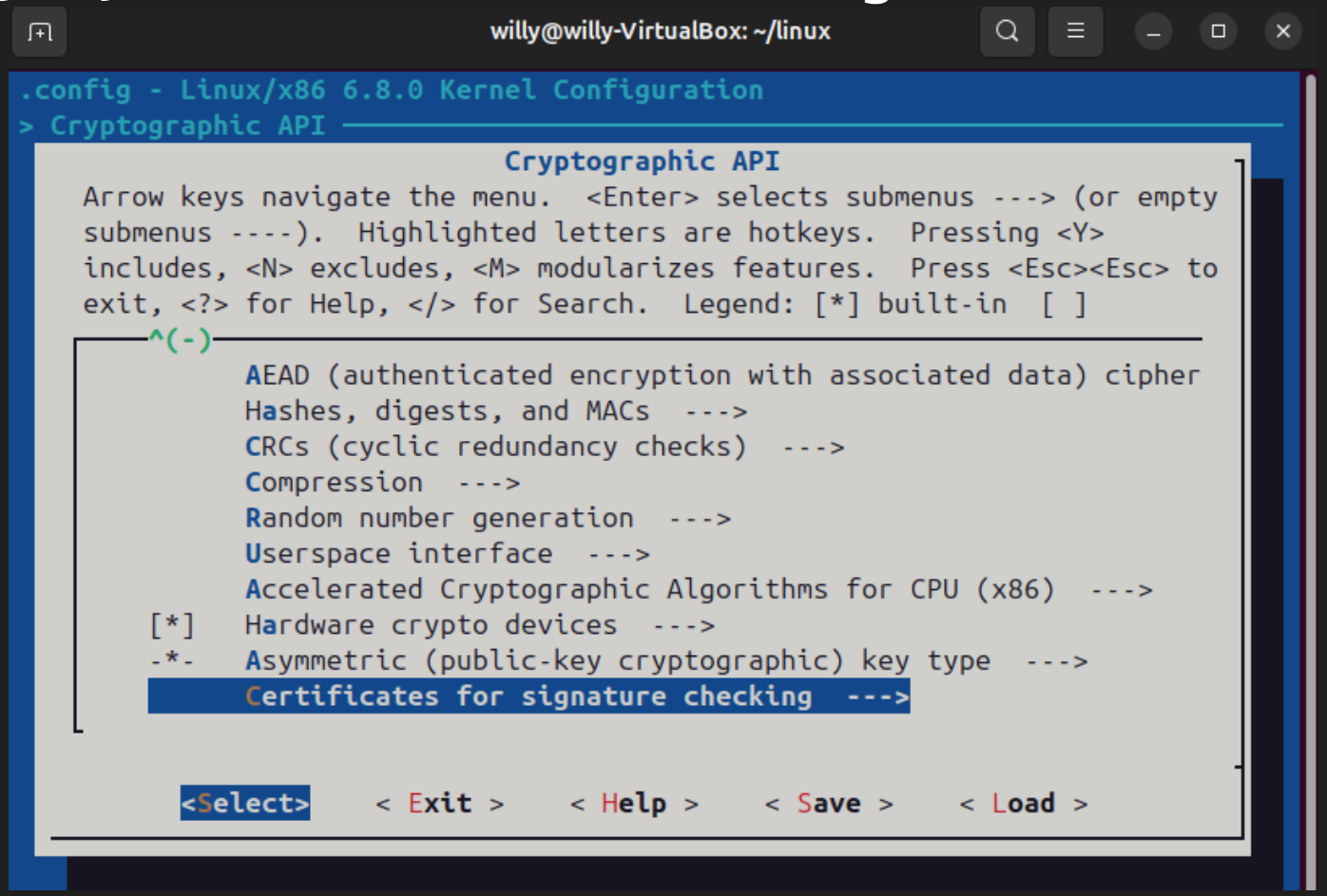
$ Compiling and installing (6/13)

- 輸入指令 make menuconfig 進到以下畫面找到 Cryptographic API



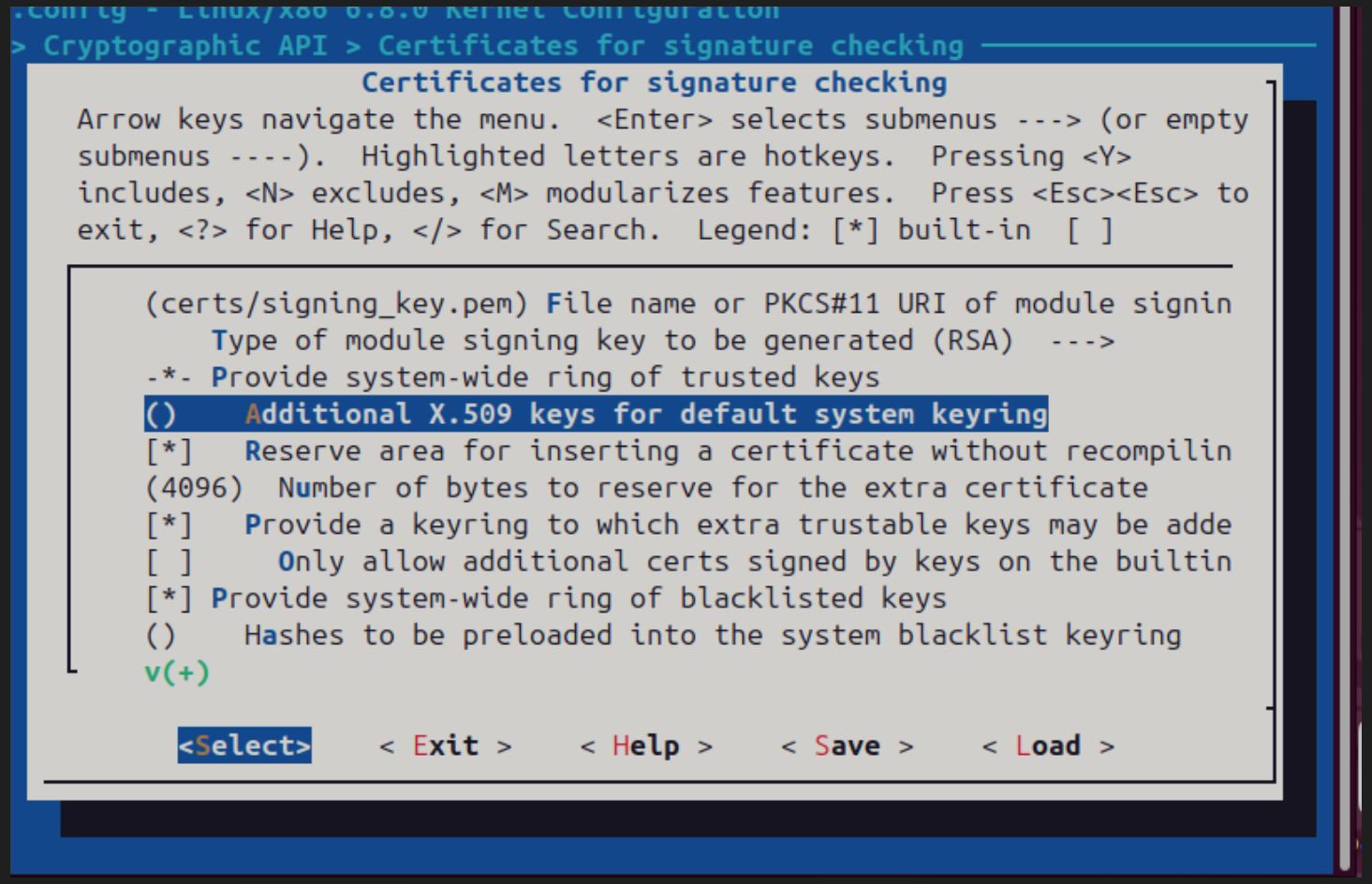
$ Compiling and installing (7/13)

- 再往下找到 Certificates for signature checking



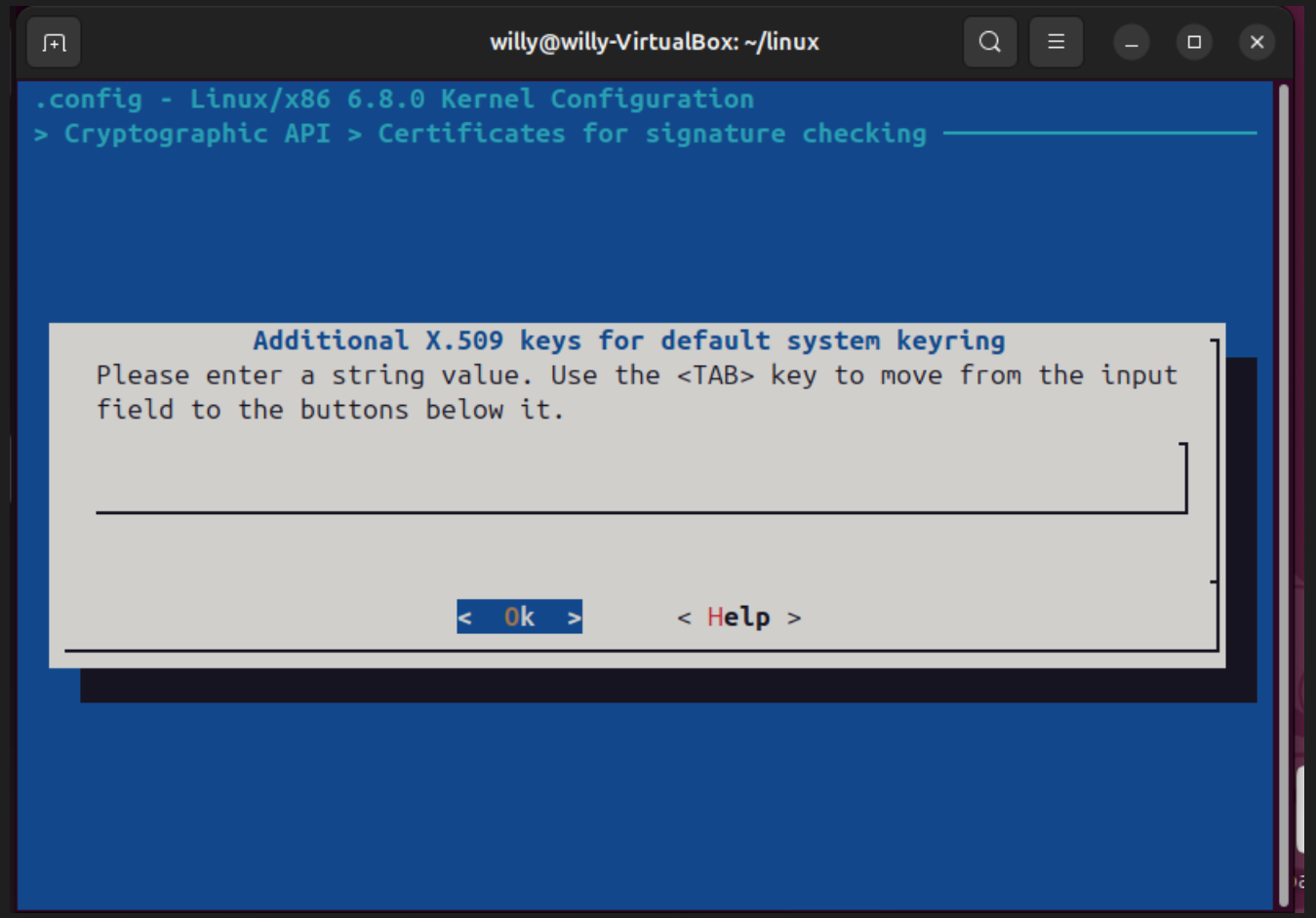
$ Compiling and installing (8/13)

- 選 additional X.509 keys for default system keyring



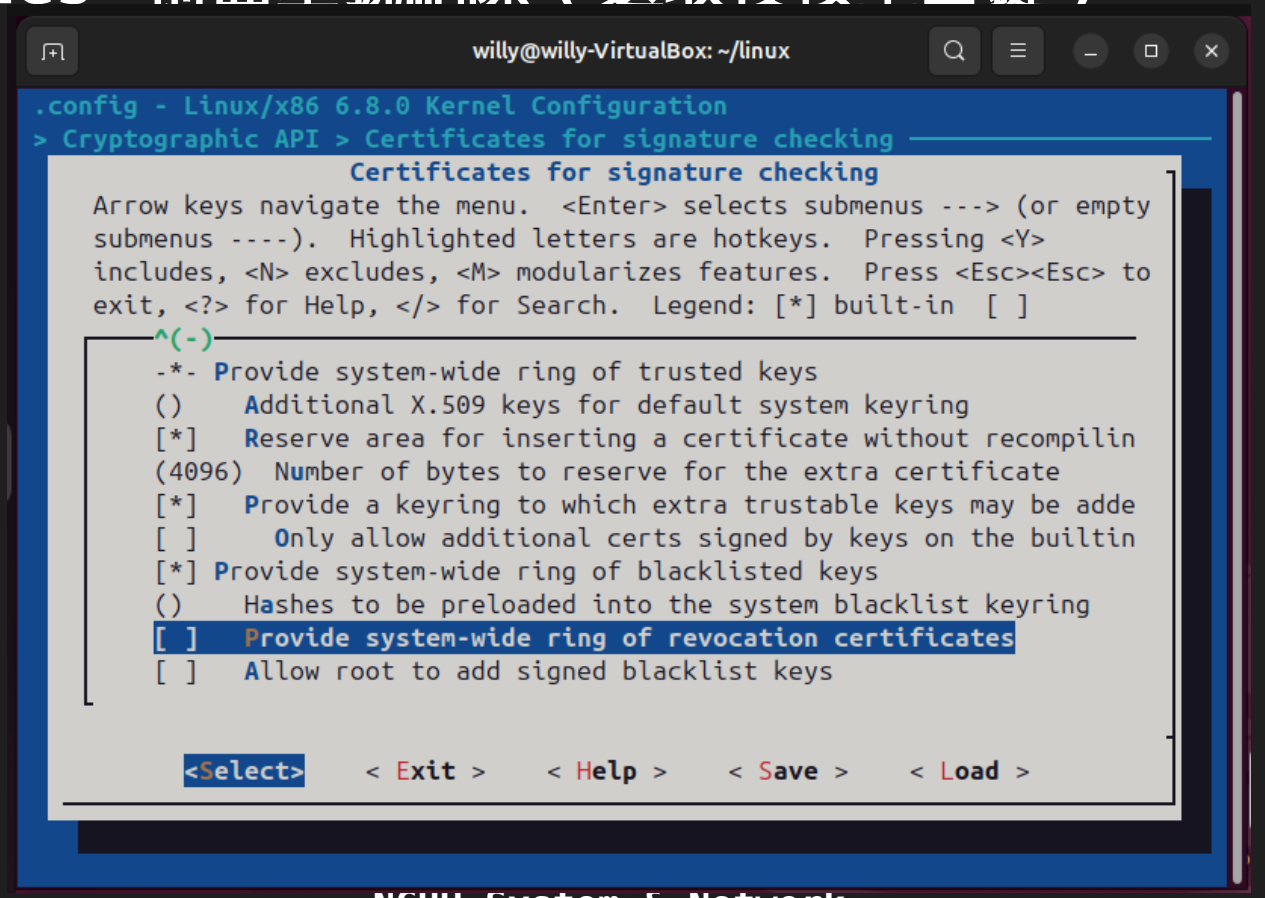
$ Compiling and installing (9/13)

- 清空路徑儲存後返回



$ Compiling and installing (10/13)

- 把 provide system-wide ring of revocation certificates 前面星號刪除（選取後按空白鍵）



$ Compiling and installing (11/13)

- 輸入 grep SYSTEM_TRUSTED_KEYS .config 應該會看到 "" 裡面路徑是空的
- 輸入 make -j $(nproc) 繼續編譯 Kernel

```
willy@willy-VirtualBox:~$ cd linux
willy@willy-VirtualBox:~/linux$ make menuconfig


*** End of the configuration.
*** Execute 'make' to start the build or try 'make help'.

willy@willy-VirtualBox:~/linux$ grep SYSTEM_TRUSTED_KEYS .config
CONFIG_SYSTEM_TRUSTED_KEYS=""
willy@willy-VirtualBox:~/linux$
```

$ Compiling and installing (12/13)

- Installing the new kernel

✔️ sudo make -j $(nproc) modules_install

```
link@link-VMware-Virtual-Platform:~/linux$ sudo make -j $(nproc) modules_install
  INSTALL /lib/modules/7.1.3-dirty/kernel/net/vmw_vsock/vsock_loopback.ko
  INSTALL /lib/modules/7.1.3-dirty/kernel/net/nsh/nsh.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/net/vmw_vsock/vmw_vsock_virtio_transport.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/net/vmw_vsock/vmw_vsock_vmci_transport.ko
  INSTALL /lib/modules/7.1.3-dirty/kernel/net/hsr/hsr.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/net/vmw_vsock/vmw_vsock_virtio_transport_common.ko
  INSTALL /lib/modules/7.1.3-dirty/kernel/net/qrtr/qrtr.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/net/vmw_vsock/hv_sock.ko
  INSTALL /lib/modules/7.1.3-dirty/kernel/net/qrtr/qrtr-smd.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/net/vmw_vsock/vsock_loopback.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/net/nsh/nsh.ko
  INSTALL /lib/modules/7.1.3-dirty/kernel/net/qrtr/qrtr-tun.ko
  INSTALL /lib/modules/7.1.3-dirty/kernel/net/qrtr/qrtr-mhi.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/net/hsr/hsr.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/net/qrtr/qrtr.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/net/qrtr/qrtr-smd.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/net/qrtr/qrtr-tun.ko
  INSTALL /lib/modules/7.1.3-dirty/kernel/virt/lib/irqbypass.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/net/qrtr/qrtr-mhi.ko
  SIGN    /lib/modules/7.1.3-dirty/kernel/virt/lib/irqbypass.ko
  DEPMOD  /lib/modules/7.1.3-dirty
```

$ Compiling and installing (13/13)

- Installing the new kernel (Cont.)

✔️ sudo make install

```
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-7.1.3-dirty
Found initrd image: /boot/initrd.img-7.1.3-dirty
Found linux image: /boot/vmlinuz-7.1.3-dirty.old
Found initrd image: /boot/initrd.img-7.1.3-dirty
Found linux image: /boot/vmlinuz-7.0.0-29-generic
Found initrd image: /boot/initrd.img-7.0.0-29-generic
Found linux image: /boot/vmlinuz-7.0.0-28-generic
Found initrd image: /boot/initrd.img-7.0.0-28-generic
Found memtest86+ 64bit EFI image: /boot/mt86+x64
Found memtest86+ 32bit EFI image: /boot/mt86+ia32
Found memtest86+ 64bit image: /boot/mt86+x64
Found memtest86+ 32bit image: /boot/mt86+ia32
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
```

$ Outline

- Introduction
- Get Linux Kernel
- Create a New Syscall
- Compiling and installing
- Testing the new syscall
- Exercise
- Reference

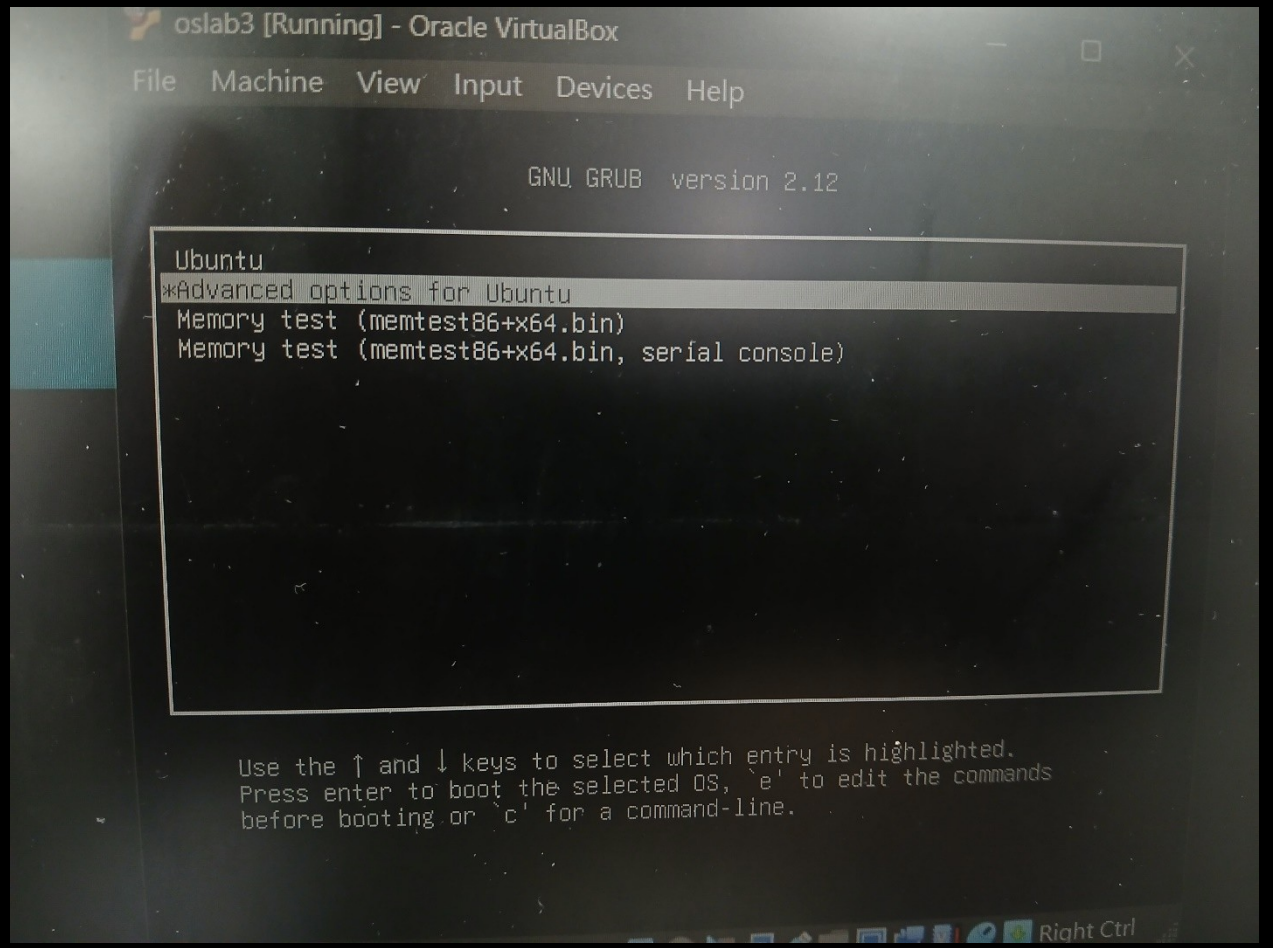
$ Testing the new syscall(1/6)

- Reboot the system, select the new custom kernel from grub
- 進入 grub 方法 ：

  重開機，在看到 Virtualbox 畫面時，點擊畫面並按住 shift

$ Testing the new syscall(2/6)

- 選 Advanced options for Ubuntu



$ Testing the new syscall(3/6)

- 選 Linux 7.1.3-dirty，即是我們加入 new syscall 的



$ Testing the new syscall(4/6)

- touch test.c
- test.c

```c
#include <stdio.h>
#include <sys/syscall.h>
#include <unistd.h>
int main(void) {
  printf("%ld\n", syscall(462));
  return 0;
}
```

$ Testing the new syscall(5/6)

- Compile the program & run
  - ✔️ make test
  - ✔️ chmod +x test
  - ✔️ ./test

```
tavha@tavha-VirtualBox:~$ cd syscall_test/
tavha@tavha-VirtualBox:~/syscall_test$ make test
make: 'test' is up to date.
tavha@tavha-VirtualBox:~/syscall_test$ chmod +x test
tavha@tavha-VirtualBox:~/syscall_test$ ./test
0
tavha@tavha-VirtualBox:~/syscall_test$
```

$ Testing the new syscall(6/6)

- sudo dmesg | tail ，查看 kernel 訊息

```
80 sp 00007ffed66f2680 error 4 in libupower-glib.so.3.1.0[7f3bb
217f000+f000] likely on CPU 2 (core 2, socket 0)
[  593.313093] Code: 8b 7d e0 ba 13 00 00 00 89 c6 e8 ab 27 ff
ff 85 c0 75 be 48 8b 45 e0 48 8d 15 34 18 00 00 be 10 00 00 00
48 8d 3d d8 05 00 00 <48> 8b 48 08 31 c0 e8 35 2c ff ff eb 9a e
8 9e 28 ff ff 66 66 2e 0f
[  604.298519] loop0: detected capacity change from 0 to 8
[  644.839722] gsd-power[3905]: segfault at 8 ip 00007f77c4303a
80 sp 00007ffd871003e0 error 4 in libupower-glib.so.3.1.0[7f77c
42f5000+f000] likely on CPU 2 (core 2, socket 0)
[  644.839737] Code: 8b 7d e0 ba 13 00 00 00 89 c6 e8 ab 27 ff
ff 85 c0 75 be 48 8b 45 e0 48 8d 15 34 18 00 00 be 10 00 00 00
48 8d 3d d8 05 00 00 <48> 8b 48 08 31 c0 e8 35 2c ff ff eb 9a e
8 9e 28 ff ff 66 66 2e 0f
[  684.324225] Hello
tavha@tavha-VirtualBox:~/syscall_test$
```

$ Outline

- Introduction
- Get Linux Kernel
- Create a New Syscall
- Compiling and installing
- Testing the new syscall
- Exercise
- Reference

$ Exercise-1 (60 pts.)

- 完成上述範例步驟，找助教 demo

```
80 sp 00007ffed66f2680 error 4 in libupower-glib.so.3.1.0[7f3bb
217f000+f000] likely on CPU 2 (core 2, socket 0)
[  593.313093] Code: 8b 7d e0 ba 13 00 00 00 89 c6 e8 ab 27 ff
ff 85 c0 75 be 48 8b 45 e0 48 8d 15 34 18 00 00 be 10 00 00 00
48 8d 3d d8 05 00 00 <48> 8b 48 08 31 c0 e8 35 2c ff ff eb 9a e
8 9e 28 ff ff 66 66 2e 0f
[  604.298519] loop0: detected capacity change from 0 to 8
[  644.839722] gsd-power[3905]: segfault at 8 ip 00007f77c4303a
80 sp 00007ffd871003e0 error 4 in libupower-glib.so.3.1.0[7f77c
42f5000+f000] likely on CPU 2 (core 2, socket 0)
[  644.839737] Code: 8b 7d e0 ba 13 00 00 00 89 c6 e8 ab 27 ff
ff 85 c0 75 be 48 8b 45 e0 48 8d 15 34 18 00 00 be 10 00 00 00
48 8d 3d d8 05 00 00 <48> 8b 48 08 31 c0 e8 35 2c ff ff eb 9a e
8 9e 28 ff ff 66 66 2e 0f
[  684.324225] Hello
tavha@tavha-VirtualBox:~/syscall_test$
```

$ Exercise-2 (1/2) (40 pts.)

- 改寫範例的 hello.c，並使用 SYSCALL_DEFINE1()，使 syscall 可接受一個參數， 傳入的參數 = 輸出 hello 的次數。
- ex: syscall(462, 3) 即為輸出 3 次 hello

```
8 9e 28 ff ff 66 66 2e 0f
[  541.824156] gsd-power[3733]: segfault at 8 ip 00007fc2b22c7a
80 sp 00007fffda223d30 error 4 in libupower-glib.so.3.1.0[7fc2b
22b9000+f000] likely on CPU 3 (core 3, socket 0)
[  541.824171] Code: 8b 7d e0 ba 13 00 00 00 89 c6 e8 ab 27 ff
ff 85 c0 75 be 48 8b 45 e0 48 8d 15 34 18 00 00 be 10 00 00 00
48 8d 3d d8 05 00 00 <48> 8b 48 08 31 c0 e8 35 2c ff ff eb 9a e
8 9e 28 ff ff 66 66 2e 0f
[  554.662505] hello
[  554.662509] hello
[  554.662510] hello
[  593.313066] gsd-power[3820]: segfault at 8 ip 00007f3bb218da
80 sp 00007ffed66f2680 error 4 in libupower-glib.so.3.1.0[7f3bb
217f000+f000] likely on CPU 2 (core 2, socket 0)
[  593.313093] Code: 8b 7d e0 ba 13 00 00 00 89 c6 e8 ab 27 ff
ff 85 c0 75 be 48 8b 45 e0 48 8d 15 34 18 00 00 be 10 00 00 00
48 8d 3d d8 05 00 00 <48> 8b 48 08 31 c0 e8 35 2c ff ff eb 9a e
8 9e 28 ff ff 66 66 2e 0f
tavha@tavha-VirtualBox:~/syscall_test2$
```

$ Exercise-2 (2/2) (40 pts.)

- 改好後需執行以下指令
- `make -j $(nproc)`
- `sudo make -j $(nproc) modules_install`
- `sudo make install`
- `reboot`

$ Reference

- https://medium.com/@aryan20/create-custom-system-call-on-linux-6-8-126edef6caaf
- https://www.linuxbnb.net/home/adding-a-system-call-to-linux-arm-architecture/



Thanks for your listening

Made by PurpleRed in 2023/08